

Module 7 Practical Exercises

This practical's notes contain the instruction of deep neural networks and recurrent neural networks using the Pytorch machine learning framework.

For deep neural networks, we will reuse the Veteran dataset from previous practicals, and compare the performance of a deep network with the single hidden layer network.

For recurrent neural networks, we will utilize long short-term memory cells to predict the maximum temperature for the next multiple timestep in various locations of Australia, given the observations from the last few days.

Preparing Data

Use the same code from previous module to load/pre-process the Veteran dataset, and split it into the training/testing set.

```
In [1]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# read the veteran dataset
df = pd.read_csv('veteran.csv')

# Change DemCluster from interval/integer to nominal/str.
# Pandas returns a copy of the data whenever you access a column, therefore we need to create a new column
df['DemCluster'] = df['DemCluster'].astype(str)

# Change DemHomeOwner into binary 0/1 variable.
dem_home_owner_map = {'U':0, 'H': 1}
df['DemHomeOwner'] = df['DemHomeOwner'].map(dem_home_owner_map)

# Set DemMedIncome to NaN for any values that are less than 1, which we will fix
mask = df['DemMedIncome'] < 1
df.loc[mask, 'DemMedIncome'] = np.nan

# Fill missing/NaN values with the mean value across the columns.
df['DemAge'] = df['DemAge'].fillna(df['DemAge'].mean())
df['DemMedIncome'] = df['DemMedIncome'].fillna(df['DemMedIncome'].mean())
df['GiftAvgCard36'] = df['GiftAvgCard36'].fillna(df['GiftAvgCard36'].mean())

# drop ID and the unused target variable
df.drop(['ID', 'TargetD'], axis=1, inplace=True)

# one-hot encoding
df = pd.get_dummies(df)

# Obtain a List of Labels (targets) for each of the samples. We refer to this as Y
Y = df["TargetB"]
```

```

# Remove the label (target) from the samples, as this is not a feature that we w
X = df.drop(['TargetB'], axis=1)

# Create a 30% train/test split from our samples.
# We set the random_state to the constant 10 to ensure it is the same from run-t
# We set the stratify parameter to the labels, that way we get an even selectio
# Otherwise, we may end up with unbalanced 0/1 labels across the two splits.
# We use X.values to get arrays for the features, instead of a DataFrame with co
random_state = 10
test_set_size = 0.3 # 30%
X_train, X_test, y_train, y_test = train_test_split(X.values, Y.values, test_siz

print("Size of training set:", len(X_train))
print("Size of testing set:", len(X_test))

random_seed = 10

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train, y_train)
X_test = scaler.transform(X_test)

```

Size of training set: 6780

Size of testing set: 2906

Deep Neural Networks

The first task of this practical exercise is to extend the multi-layer perceptron neural network to a greater number of layers, creating a deep neural deep network. The "deep" aspect refers to having many hidden layers between input and output, allowing these systems to automatically discover intricate relationships within data without explicit programming of rules.

You will need Pytorch installed and configured for this practical, refer to previous modules for instructions on how to install Pytorch.

First, we will convert the training and test datasets into `torch.tensor`, making them compatible with Pytorch.

```

In [2]: import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
from sklearn.metrics import accuracy_score

X_train = torch.tensor(X_train, dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.float32).reshape(-1, 1)
X_test = torch.tensor(X_test, dtype=torch.float32)
y_test = torch.tensor(y_test, dtype=torch.float32).reshape(-1, 1)

```

Next, we will define the network itself.

Pytorch contains the `Sequential` class, which provides a simplified way to build neural networks by arranging layers in a linear sequence, where the output of one layer serves as the input to the next. This container allows developers to define a network's

architecture as an ordered list of modules that are automatically connected in sequence, eliminating the need to manually specify how data flows between components.

```
In [3]: num_features = X_train.shape[1]
hidden_layer_size = 25
output_size = 1
n_epochs = 100
lr = 0.001
# define the model
model = nn.Sequential(
    nn.Linear(num_features, hidden_layer_size),
    nn.Linear(hidden_layer_size, output_size),
    nn.Sigmoid()
)
print(model)
```

```
Sequential(
  (0): Linear(in_features=85, out_features=25, bias=True)
  (1): Linear(in_features=25, out_features=1, bias=True)
  (2): Sigmoid()
)
```

You will notice that we are explicitly passing the data from one layer to the next, after applying the activation function. The `first` layer is followed by the ReLU activation function, followed by the `second` layer, which maps the hidden layer to the output size, which in this case is a single numerical value.

For our deep neural network, we will use the `Sequential` class. The straightforward syntax makes it particularly useful for quickly prototyping linear network structures, though more complex architectures with branching or skip connections would require custom Module implementations instead.

Let's start by defining a deep network with 5 hidden layers. Each of these hidden layers is represented by a `Linear` class in Pytorch, and we apply the ReLU activation function after each of them. We will use the same size for each of the hidden layers, but this is not required.

```
In [4]: num_features = X_train.shape[1]
hidden_layer_size = 25
output_size = 1
in_features = num_features
# define the deep model
deep_model = nn.Sequential(
    nn.Linear(in_features, hidden_layer_size), # Input Layer -> First Hi
    nn.ReLU(), # Activation Function
    nn.Linear(hidden_layer_size, hidden_layer_size), # First Hidden Layer
    nn.ReLU(), # Activation Function
    nn.Linear(hidden_layer_size, hidden_layer_size), # Second Hidden Layer
    nn.ReLU(), # Activation Function
    nn.Linear(hidden_layer_size, hidden_layer_size), # Third Hidden Layer
    nn.ReLU(), # Activation Function
    nn.Linear(hidden_layer_size, output_size), # Fourth Hidden Layer ->
    nn.Sigmoid() # Activation Function
)
print(deep_model)
```

```

Sequential(
  (0): Linear(in_features=85, out_features=25, bias=True)
  (1): ReLU()
  (2): Linear(in_features=25, out_features=25, bias=True)
  (3): ReLU()
  (4): Linear(in_features=25, out_features=25, bias=True)
  (5): ReLU()
  (6): Linear(in_features=25, out_features=25, bias=True)
  (7): ReLU()
  (8): Linear(in_features=25, out_features=1, bias=True)
  (9): Sigmoid()
)

```

Now we can train and evaluate both networks by passing them into

`train_predict_model` function.

```

In [5]: def train_predict_model(model, n_epochs, lr, X_train, y_train, X_test, y_test):
        # train the model
        loss_fn = nn.BCELoss() # binary cross entropy
        optimizer = optim.Adam(model.parameters(), lr=lr)
        count = sum(p.numel() for p in model.parameters() if p.requires_grad)
        print(f"The model has {count:,} trainable parameters")

        train_acc_history = []
        test_acc_history = []
        for i in range(n_epochs):
            y_pred = model(X_train)
            loss = loss_fn(y_pred, y_train)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()
            with torch.no_grad():
                train_pred = model(X_train)
                train_pred = (train_pred >= 0.5).float()
                train_acc = accuracy_score(y_train, train_pred)
                test_pred = model(X_test)
                test_pred = (test_pred >= 0.5).float()
                test_acc = accuracy_score(y_test, test_pred)
            if (i % 1) == 0:
                print(f"Epoch {i+1}: Train Accuracy: {train_acc} Test Accuracy: {tes
                train_acc_history.append(train_acc)
                test_acc_history.append(test_acc)
        return train_acc_history, test_acc_history

```

```

In [6]: model_train_acc, model_test_acc = train_predict_model(model, n_epochs, lr, X_train, y_train, X_test, y_test)
        deep_model_train_acc, deep_model_test_acc = train_predict_model(deep_model, n_epochs, lr, X_train, y_train, X_test, y_test)

```

The model has 2,176 trainable parameters

Epoch 1: Train Accuracy: 0.5076696165191741 Test Accuracy: 0.5141087405368203
Epoch 2: Train Accuracy: 0.5157817109144542 Test Accuracy: 0.5206469373709567
Epoch 3: Train Accuracy: 0.5215339233038349 Test Accuracy: 0.5233998623537509
Epoch 4: Train Accuracy: 0.5311209439528024 Test Accuracy: 0.5344115622849277
Epoch 5: Train Accuracy: 0.53952802359882 Test Accuracy: 0.5378527185134205
Epoch 6: Train Accuracy: 0.5463126843657817 Test Accuracy: 0.5412938747419133
Epoch 7: Train Accuracy: 0.551622418879056 Test Accuracy: 0.5447350309704061
Epoch 8: Train Accuracy: 0.5570796460176991 Test Accuracy: 0.5495526496902959
Epoch 9: Train Accuracy: 0.5601769911504425 Test Accuracy: 0.550929112181693
Epoch 10: Train Accuracy: 0.5641592920353983 Test Accuracy: 0.5536820371644873
Epoch 11: Train Accuracy: 0.5666666666666667 Test Accuracy: 0.550929112181693
Epoch 12: Train Accuracy: 0.5688790560471977 Test Accuracy: 0.5564349621472815
Epoch 13: Train Accuracy: 0.5734513274336284 Test Accuracy: 0.5557467309015829
Epoch 14: Train Accuracy: 0.5765486725663717 Test Accuracy: 0.5557467309015829
Epoch 15: Train Accuracy: 0.5771386430678466 Test Accuracy: 0.5554026152787337
Epoch 16: Train Accuracy: 0.578023598820059 Test Accuracy: 0.5560908465244322
Epoch 17: Train Accuracy: 0.5783185840707965 Test Accuracy: 0.5588437715072264
Epoch 18: Train Accuracy: 0.58023598820059 Test Accuracy: 0.5591878871300757
Epoch 19: Train Accuracy: 0.5800884955752212 Test Accuracy: 0.5622849277357193
Epoch 20: Train Accuracy: 0.5790560471976401 Test Accuracy: 0.5602202339986235
Epoch 21: Train Accuracy: 0.5809734513274336 Test Accuracy: 0.5605643496214728
Epoch 22: Train Accuracy: 0.5812684365781711 Test Accuracy: 0.559532002752925
Epoch 23: Train Accuracy: 0.5828908554572272 Test Accuracy: 0.5605643496214728
Epoch 24: Train Accuracy: 0.5823008849557522 Test Accuracy: 0.5584996558843771
Epoch 25: Train Accuracy: 0.5805309734513274 Test Accuracy: 0.5584996558843771
Epoch 26: Train Accuracy: 0.5833333333333334 Test Accuracy: 0.5598761183757742
Epoch 27: Train Accuracy: 0.584070796460177 Test Accuracy: 0.5591878871300757
Epoch 28: Train Accuracy: 0.5842182890855457 Test Accuracy: 0.5578114246386786
Epoch 29: Train Accuracy: 0.5837758112094396 Test Accuracy: 0.5605643496214728
Epoch 30: Train Accuracy: 0.5843657817109145 Test Accuracy: 0.559532002752925
Epoch 31: Train Accuracy: 0.5837758112094396 Test Accuracy: 0.5598761183757742
Epoch 32: Train Accuracy: 0.585693215339233 Test Accuracy: 0.5605643496214728
Epoch 33: Train Accuracy: 0.5874631268436579 Test Accuracy: 0.5622849277357193
Epoch 34: Train Accuracy: 0.5880530973451328 Test Accuracy: 0.5602202339986235
Epoch 35: Train Accuracy: 0.5889380530973451 Test Accuracy: 0.5598761183757742
Epoch 36: Train Accuracy: 0.5895280235988201 Test Accuracy: 0.5598761183757742
Epoch 37: Train Accuracy: 0.5905604719764012 Test Accuracy: 0.559532002752925
Epoch 38: Train Accuracy: 0.590117994100295 Test Accuracy: 0.5609084652443221
Epoch 39: Train Accuracy: 0.5904129793510324 Test Accuracy: 0.5598761183757742
Epoch 40: Train Accuracy: 0.5904129793510324 Test Accuracy: 0.5581555402615279
Epoch 41: Train Accuracy: 0.5921828908554573 Test Accuracy: 0.5588437715072264
Epoch 42: Train Accuracy: 0.5932153392330384 Test Accuracy: 0.5612525808671713
Epoch 43: Train Accuracy: 0.5921828908554573 Test Accuracy: 0.5615966964900206
Epoch 44: Train Accuracy: 0.5907079646017699 Test Accuracy: 0.5626290433585684
Epoch 45: Train Accuracy: 0.591740412979351 Test Accuracy: 0.5629731589814178
Epoch 46: Train Accuracy: 0.591740412979351 Test Accuracy: 0.56194081211287
Epoch 47: Train Accuracy: 0.5910029498525073 Test Accuracy: 0.5640055058499656
Epoch 48: Train Accuracy: 0.5910029498525073 Test Accuracy: 0.5626290433585684
Epoch 49: Train Accuracy: 0.5910029498525073 Test Accuracy: 0.5640055058499656
Epoch 50: Train Accuracy: 0.5910029498525073 Test Accuracy: 0.5629731589814178
Epoch 51: Train Accuracy: 0.5905604719764012 Test Accuracy: 0.5640055058499656
Epoch 52: Train Accuracy: 0.5915929203539823 Test Accuracy: 0.5636613902271164
Epoch 53: Train Accuracy: 0.5905604719764012 Test Accuracy: 0.5646937370956642
Epoch 54: Train Accuracy: 0.5905604719764012 Test Accuracy: 0.5650378527185134
Epoch 55: Train Accuracy: 0.5914454277286135 Test Accuracy: 0.5650378527185134
Epoch 56: Train Accuracy: 0.5920353982300885 Test Accuracy: 0.5667584308327598
Epoch 57: Train Accuracy: 0.5920353982300885 Test Accuracy: 0.5660701995870613
Epoch 58: Train Accuracy: 0.5930678466076696 Test Accuracy: 0.5660701995870613
Epoch 59: Train Accuracy: 0.5938053097345133 Test Accuracy: 0.5643496214728149

Epoch 60: Train Accuracy: 0.5945427728613569 Test Accuracy: 0.5640055058499656
Epoch 61: Train Accuracy: 0.5948377581120944 Test Accuracy: 0.5636613902271164
Epoch 62: Train Accuracy: 0.5954277286135693 Test Accuracy: 0.5633172746042671
Epoch 63: Train Accuracy: 0.5948377581120944 Test Accuracy: 0.5629731589814178
Epoch 64: Train Accuracy: 0.595575221238938 Test Accuracy: 0.5629731589814178
Epoch 65: Train Accuracy: 0.5942477876106195 Test Accuracy: 0.5626290433585684
Epoch 66: Train Accuracy: 0.5938053097345133 Test Accuracy: 0.56194081211287
Epoch 67: Train Accuracy: 0.5927728613569322 Test Accuracy: 0.5609084652443221
Epoch 68: Train Accuracy: 0.5935103244837758 Test Accuracy: 0.5598761183757742
Epoch 69: Train Accuracy: 0.5945427728613569 Test Accuracy: 0.5598761183757742
Epoch 70: Train Accuracy: 0.5951327433628318 Test Accuracy: 0.5612525808671713
Epoch 71: Train Accuracy: 0.5949852507374631 Test Accuracy: 0.559532002752925
Epoch 72: Train Accuracy: 0.5949852507374631 Test Accuracy: 0.5591878871300757
Epoch 73: Train Accuracy: 0.5949852507374631 Test Accuracy: 0.559532002752925
Epoch 74: Train Accuracy: 0.5951327433628318 Test Accuracy: 0.559532002752925
Epoch 75: Train Accuracy: 0.5946902654867257 Test Accuracy: 0.5598761183757742
Epoch 76: Train Accuracy: 0.5945427728613569 Test Accuracy: 0.5598761183757742
Epoch 77: Train Accuracy: 0.5949852507374631 Test Accuracy: 0.5602202339986235
Epoch 78: Train Accuracy: 0.5948377581120944 Test Accuracy: 0.5598761183757742
Epoch 79: Train Accuracy: 0.5952802359882006 Test Accuracy: 0.559532002752925
Epoch 80: Train Accuracy: 0.5954277286135693 Test Accuracy: 0.5591878871300757
Epoch 81: Train Accuracy: 0.5954277286135693 Test Accuracy: 0.5598761183757742
Epoch 82: Train Accuracy: 0.5958702064896755 Test Accuracy: 0.5605643496214728
Epoch 83: Train Accuracy: 0.595575221238938 Test Accuracy: 0.5609084652443221
Epoch 84: Train Accuracy: 0.5952802359882006 Test Accuracy: 0.5609084652443221
Epoch 85: Train Accuracy: 0.5954277286135693 Test Accuracy: 0.5609084652443221
Epoch 86: Train Accuracy: 0.5954277286135693 Test Accuracy: 0.5605643496214728
Epoch 87: Train Accuracy: 0.5951327433628318 Test Accuracy: 0.5609084652443221
Epoch 88: Train Accuracy: 0.5946902654867257 Test Accuracy: 0.5615966964900206
Epoch 89: Train Accuracy: 0.5946902654867257 Test Accuracy: 0.5615966964900206
Epoch 90: Train Accuracy: 0.5943952802359882 Test Accuracy: 0.5622849277357193
Epoch 91: Train Accuracy: 0.593952802359882 Test Accuracy: 0.5629731589814178
Epoch 92: Train Accuracy: 0.5938053097345133 Test Accuracy: 0.5636613902271164
Epoch 93: Train Accuracy: 0.5933628318584071 Test Accuracy: 0.5640055058499656
Epoch 94: Train Accuracy: 0.5933628318584071 Test Accuracy: 0.5643496214728149
Epoch 95: Train Accuracy: 0.5935103244837758 Test Accuracy: 0.5646937370956642
Epoch 96: Train Accuracy: 0.5936578171091446 Test Accuracy: 0.5650378527185134
Epoch 97: Train Accuracy: 0.593952802359882 Test Accuracy: 0.5650378527185134
Epoch 98: Train Accuracy: 0.5936578171091446 Test Accuracy: 0.5653819683413627
Epoch 99: Train Accuracy: 0.593952802359882 Test Accuracy: 0.5650378527185134
Epoch 100: Train Accuracy: 0.5938053097345133 Test Accuracy: 0.5650378527185134
The model has 4,126 trainable parameters
Epoch 1: Train Accuracy: 0.49985250737463127 Test Accuracy: 0.5
Epoch 2: Train Accuracy: 0.5 Test Accuracy: 0.49965588437715075
Epoch 3: Train Accuracy: 0.5001474926253687 Test Accuracy: 0.49965588437715075
Epoch 4: Train Accuracy: 0.5011799410029498 Test Accuracy: 0.4986235375086029
Epoch 5: Train Accuracy: 0.5036873156342183 Test Accuracy: 0.4986235375086029
Epoch 6: Train Accuracy: 0.5058997050147492 Test Accuracy: 0.5024088093599449
Epoch 7: Train Accuracy: 0.5166666666666667 Test Accuracy: 0.5027529249827942
Epoch 8: Train Accuracy: 0.5253687315634218 Test Accuracy: 0.5110116999311769
Epoch 9: Train Accuracy: 0.5358407079646018 Test Accuracy: 0.5182381280110117

Epoch 10: Train Accuracy: 0.5502949852507375 Test Accuracy: 0.5271851342050929
Epoch 11: Train Accuracy: 0.5582595870206489 Test Accuracy: 0.5371644872677219
Epoch 12: Train Accuracy: 0.5613569321533923 Test Accuracy: 0.5474879559532003
Epoch 13: Train Accuracy: 0.5617994100294985 Test Accuracy: 0.5406056434962148
Epoch 14: Train Accuracy: 0.5616519174041298 Test Accuracy: 0.5440467997247075
Epoch 15: Train Accuracy: 0.5572271386430678 Test Accuracy: 0.5495526496902959
Epoch 16: Train Accuracy: 0.5585545722713864 Test Accuracy: 0.5478320715760495
Epoch 17: Train Accuracy: 0.5573746312684366 Test Accuracy: 0.5481761871988988
Epoch 18: Train Accuracy: 0.5582595870206489 Test Accuracy: 0.5523055746730902
Epoch 19: Train Accuracy: 0.557669616519174 Test Accuracy: 0.5536820371644873
Epoch 20: Train Accuracy: 0.5591445427728614 Test Accuracy: 0.5526496902959395
Epoch 21: Train Accuracy: 0.5612094395280236 Test Accuracy: 0.5516173434273915
Epoch 22: Train Accuracy: 0.5616519174041298 Test Accuracy: 0.5519614590502409
Epoch 23: Train Accuracy: 0.5617994100294985 Test Accuracy: 0.5502408809359945
Epoch 24: Train Accuracy: 0.5631268436578171 Test Accuracy: 0.5550584996558844
Epoch 25: Train Accuracy: 0.567551622418879 Test Accuracy: 0.5529938059187887
Epoch 26: Train Accuracy: 0.5690265486725664 Test Accuracy: 0.5567790777701308
Epoch 27: Train Accuracy: 0.5722713864306784 Test Accuracy: 0.5564349621472815
Epoch 28: Train Accuracy: 0.5738938053097346 Test Accuracy: 0.5581555402615279
Epoch 29: Train Accuracy: 0.5766961651917404 Test Accuracy: 0.559532002752925
Epoch 30: Train Accuracy: 0.5766961651917404 Test Accuracy: 0.5605643496214728
Epoch 31: Train Accuracy: 0.5789085545722714 Test Accuracy: 0.559532002752925
Epoch 32: Train Accuracy: 0.5821533923303834 Test Accuracy: 0.5588437715072264
Epoch 33: Train Accuracy: 0.5839233038348083 Test Accuracy: 0.5588437715072264
Epoch 34: Train Accuracy: 0.5842182890855457 Test Accuracy: 0.55712319339298
Epoch 35: Train Accuracy: 0.5853982300884956 Test Accuracy: 0.5529938059187887
Epoch 36: Train Accuracy: 0.5873156342182891 Test Accuracy: 0.553337921541638
Epoch 37: Train Accuracy: 0.588495575221239 Test Accuracy: 0.5550584996558844
Epoch 38: Train Accuracy: 0.5880530973451328 Test Accuracy: 0.5588437715072264
Epoch 39: Train Accuracy: 0.5907079646017699 Test Accuracy: 0.553337921541638
Epoch 40: Train Accuracy: 0.5921828908554573 Test Accuracy: 0.5536820371644873
Epoch 41: Train Accuracy: 0.593952802359882 Test Accuracy: 0.5529938059187887
Epoch 42: Train Accuracy: 0.5938053097345133 Test Accuracy: 0.55712319339298
Epoch 43: Train Accuracy: 0.5964601769911504 Test Accuracy: 0.5564349621472815
Epoch 44: Train Accuracy: 0.5954277286135693 Test Accuracy: 0.5591878871300757
Epoch 45: Train Accuracy: 0.5961651917404129 Test Accuracy: 0.5602202339986235
Epoch 46: Train Accuracy: 0.5973451327433629 Test Accuracy: 0.5602202339986235
Epoch 47: Train Accuracy: 0.5960176991150442 Test Accuracy: 0.559532002752925
Epoch 48: Train Accuracy: 0.5971976401179941 Test Accuracy: 0.5584996558843771
Epoch 49: Train Accuracy: 0.5969026548672567 Test Accuracy: 0.5574673090158293
Epoch 50: Train Accuracy: 0.5995575221238938 Test Accuracy: 0.5578114246386786
Epoch 51: Train Accuracy: 0.6007374631268436 Test Accuracy: 0.5578114246386786
Epoch 52: Train Accuracy: 0.6028023598820059 Test Accuracy: 0.5591878871300757
Epoch 53: Train Accuracy: 0.6041297935103245 Test Accuracy: 0.5615966964900206
Epoch 54: Train Accuracy: 0.6057522123893805 Test Accuracy: 0.5636613902271164
Epoch 55: Train Accuracy: 0.6070796460176991 Test Accuracy: 0.5643496214728149
Epoch 56: Train Accuracy: 0.608259587020649 Test Accuracy: 0.5667584308327598
Epoch 57: Train Accuracy: 0.6084070796460177 Test Accuracy: 0.5660701995870613
Epoch 58: Train Accuracy: 0.6103244837758112 Test Accuracy: 0.5646937370956642
Epoch 59: Train Accuracy: 0.6106194690265486 Test Accuracy: 0.5650378527185134
Epoch 60: Train Accuracy: 0.6126843657817109 Test Accuracy: 0.5660701995870613
Epoch 61: Train Accuracy: 0.6131268436578171 Test Accuracy: 0.5653819683413627
Epoch 62: Train Accuracy: 0.6147492625368731 Test Accuracy: 0.5643496214728149
Epoch 63: Train Accuracy: 0.6148967551622418 Test Accuracy: 0.5626290433585684
Epoch 64: Train Accuracy: 0.6154867256637168 Test Accuracy: 0.5633172746042671
Epoch 65: Train Accuracy: 0.6175516224188791 Test Accuracy: 0.5626290433585684
Epoch 66: Train Accuracy: 0.6188790560471976 Test Accuracy: 0.5633172746042671
Epoch 67: Train Accuracy: 0.6213864306784661 Test Accuracy: 0.5605643496214728
Epoch 68: Train Accuracy: 0.6228613569321534 Test Accuracy: 0.5602202339986235
Epoch 69: Train Accuracy: 0.6228613569321534 Test Accuracy: 0.5588437715072264

```

Epoch 70: Train Accuracy: 0.6255162241887906 Test Accuracy: 0.5588437715072264
Epoch 71: Train Accuracy: 0.6274336283185841 Test Accuracy: 0.55712319339298
Epoch 72: Train Accuracy: 0.628023598820059 Test Accuracy: 0.5574673090158293
Epoch 73: Train Accuracy: 0.6293510324483775 Test Accuracy: 0.5554026152787337
Epoch 74: Train Accuracy: 0.6308259587020649 Test Accuracy: 0.5564349621472815
Epoch 75: Train Accuracy: 0.6309734513274337 Test Accuracy: 0.5567790777701308
Epoch 76: Train Accuracy: 0.6323008849557522 Test Accuracy: 0.5574673090158293
Epoch 77: Train Accuracy: 0.6323008849557522 Test Accuracy: 0.5557467309015829
Epoch 78: Train Accuracy: 0.6339233038348082 Test Accuracy: 0.5560908465244322
Epoch 79: Train Accuracy: 0.6358407079646018 Test Accuracy: 0.5574673090158293
Epoch 80: Train Accuracy: 0.6351032448377582 Test Accuracy: 0.5578114246386786
Epoch 81: Train Accuracy: 0.6373156342182891 Test Accuracy: 0.5581555402615279
Epoch 82: Train Accuracy: 0.6384955752212389 Test Accuracy: 0.5584996558843771
Epoch 83: Train Accuracy: 0.6423303834808259 Test Accuracy: 0.5574673090158293
Epoch 84: Train Accuracy: 0.6441002949852508 Test Accuracy: 0.56194081211287
Epoch 85: Train Accuracy: 0.6439528023598821 Test Accuracy: 0.5612525808671713
Epoch 86: Train Accuracy: 0.6455752212389381 Test Accuracy: 0.5591878871300757
Epoch 87: Train Accuracy: 0.6470501474926253 Test Accuracy: 0.559532002752925
Epoch 88: Train Accuracy: 0.6479351032448377 Test Accuracy: 0.5612525808671713
Epoch 89: Train Accuracy: 0.6486725663716815 Test Accuracy: 0.56194081211287
Epoch 90: Train Accuracy: 0.6511799410029498 Test Accuracy: 0.5612525808671713
Epoch 91: Train Accuracy: 0.6530973451327433 Test Accuracy: 0.56194081211287
Epoch 92: Train Accuracy: 0.6539823008849558 Test Accuracy: 0.5602202339986235
Epoch 93: Train Accuracy: 0.6550147492625369 Test Accuracy: 0.5591878871300757
Epoch 94: Train Accuracy: 0.6566371681415929 Test Accuracy: 0.5560908465244322
Epoch 95: Train Accuracy: 0.6588495575221239 Test Accuracy: 0.5526496902959395
Epoch 96: Train Accuracy: 0.6606194690265487 Test Accuracy: 0.5529938059187887
Epoch 97: Train Accuracy: 0.6617994100294985 Test Accuracy: 0.5540261527873366
Epoch 98: Train Accuracy: 0.6637168141592921 Test Accuracy: 0.553337921541638
Epoch 99: Train Accuracy: 0.6675516224188791 Test Accuracy: 0.5516173434273915
Epoch 100: Train Accuracy: 0.6703539823008849 Test Accuracy: 0.5498967653131452

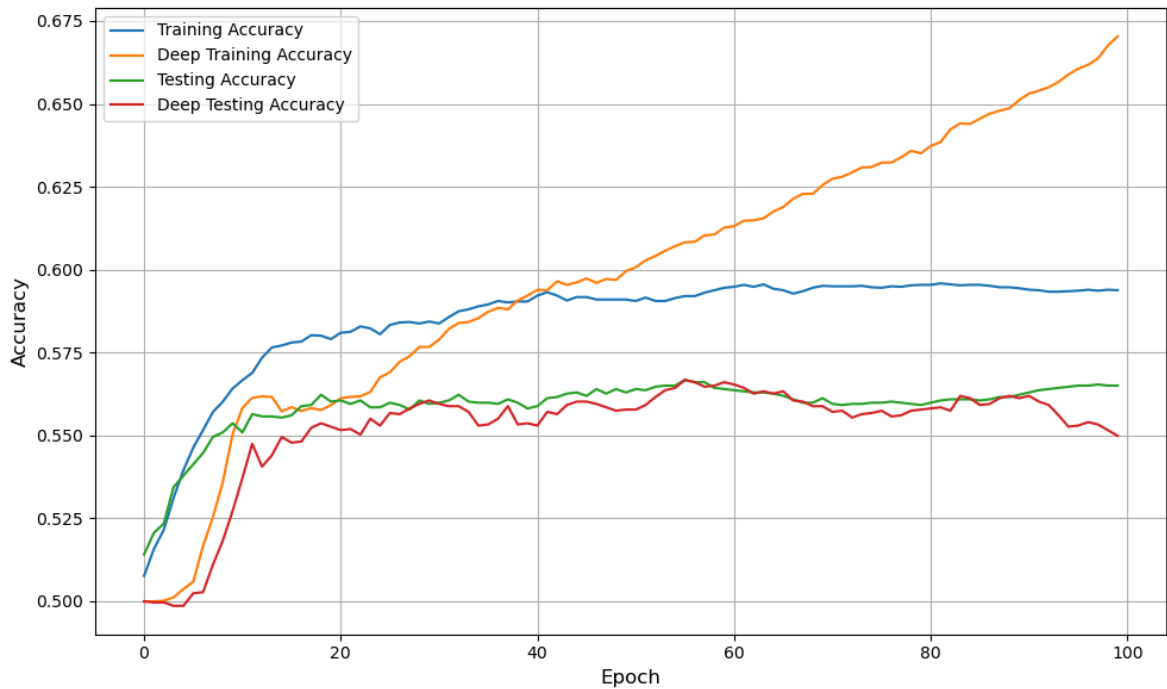
```

Plot and compare the shallow network with a single hidden layer to our deep network with four hidden layers:

```

In [7]: import matplotlib.pyplot as plt
plt.figure(figsize=(10, 6))
plt.xlabel("Epoch", size=12)
plt.ylabel("Accuracy", size=12)
plt.plot(model_train_acc, label="Training Accuracy")
plt.plot(deep_model_train_acc, label="Deep Training Accuracy")
plt.plot(model_test_acc, label="Testing Accuracy")
plt.plot(deep_model_test_acc, label="Deep Testing Accuracy")
plt.grid()
plt.legend(fontsize=10)
plt.tight_layout()
plt.show()

```



Comparing the parameter count, we can see that the shallow network has `~4,300` trainable parameters, whereas the deep network has `~12,000`. This suggests that the deep network has significantly more capacity for learning, which also risks overfitting.

We can see this overfitting by examining the plot above: the deep network achieved a much greater training accuracy of almost 90% after 100 epochs, compared to the shallow network which only reached ~65% accuracy after the same amount of time. However, this resulted in overfitting after around epoch #30, with the training accuracy continuing to increase, but the testing accuracy starting to drop.

Therefore, if we were deploying this model, we would want to cease training the network after around 30 epochs. Using early stopping would be one method for automatically stopping training after a number of consecutive epochs resulted in reduced testing accuracy, and roll back to the last "good" set of weights.

We can also observe that the deep network achieved slightly higher testing accuracy at its peak compared to the shallow network. At epoch #39, the accuracy of the deep network was around 56-57%, while the shallow network was 55-56%.

Recurrent Neural Networks

In this module, we introduced the concept of recurrent neural networks. Specifically, we introduced the long short-term memory (LSTM) neural network, which we will be demonstrating in this practical session.

Long Short-Term Memory (LSTM) networks are a specialized form of recurrent neural networks designed to overcome the vanishing gradient problem that hampers learning in traditional RNNs. By utilizing a gating mechanism that consists of input, forget and output gates, LSTMs can selectively remember or discard information, allowing them to capture long-range dependencies in sequential data. Each LSTM cell maintains a cell

state that acts as a conveyor belt of information flowing through the sequence, with gates controlling what information is added, removed, or passed to the output.

LSTMs have a range of applications, including natural language processing, speech recognition, time series prediction, and any domain where contextual information across varying time scales is crucial for accurate predictions. In this practical, we will be exploring time series prediction using LSTMs. While the implementation details for other applications will differ, the high-level approach is the same.

Dataset for our LSTM network

In this practical, we will use a subset of Australian weather data that contains observations from many locations across Australia, including the minimum/maximum temperature, rainfall, wind, humidity, and pressure. The dataset was preprocessed and contains ~10,000 rows from different cities, and the following columns:

- Location - Name of the city from Australia.
- MinTemp - The Minimum temperature during a particular day. (degree Celsius)
- MaxTemp - The maximum temperature during a particular day. (degree Celsius)
- Rainfall - Rainfall during a particular day. (millimeters)
- WindGuSpeed - Speed of strongest gust during a particular day. (kilometers per hour)
- WindSpeed9am - Speed of the wind for 10 min prior to 9 am. (kilometers per hour)
- WindSpeed3pm - Speed of the wind for 10 min prior to 3 pm. (kilometers per hour)
- Humidity9am - The humidity of the wind at 9 am. (percent)
- Humidity3pm - The humidity of the wind at 3 pm. (percent)
- Pressure9am - Atmospheric pressure at 9 am. (hectopascals)
- Pressure3pm - Atmospheric pressure at 3 pm. (hectopascals)
- Temp9am - The temperature at 9 am. (degree Celsius)
- Temp3pm - The temperature at 3 pm. (degree Celsius)

This raw dataset and information was obtained from Kaggle:

<https://www.kaggle.com/datasets/arunavakrchakraborty/australia-weather-data>

The observations in the dataset itself were sourced from the Bureau of Meteorology, and collated by Arunava Kr. Chakraborty.

Loading and cleaning the data

We will load the dataset using Pandas as we have with other datasets in the other practicals.

This dataset contains invalid values in some rows, which end up as NaN values. We should remove these rows using the `dropna()` function.

Our goal for this LSTM model is to predict the maximum temperature, given the last 5 days of data.

```
In [8]: df = pd.read_csv("weather.csv")
```

```
In [9]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 99516 entries, 0 to 99515
Data columns (total 23 columns):
#   Column                Non-Null Count  Dtype
---  -
0   row ID                99516 non-null  object
1   Location              99516 non-null  object
2   MinTemp              99073 non-null  float64
3   MaxTemp              99286 non-null  float64
4   Rainfall             98537 non-null  float64
5   Evaporation          56985 non-null  float64
6   Sunshine             52199 non-null  float64
7   WindGustDir          92995 non-null  object
8   WindGustSpeed        93036 non-null  float64
9   WindDir9am           92510 non-null  object
10  WindDir3pm           96868 non-null  object
11  WindSpeed9am         98581 non-null  float64
12  WindSpeed3pm         97681 non-null  float64
13  Humidity9am          98283 non-null  float64
14  Humidity3pm          97010 non-null  float64
15  Pressure9am          89768 non-null  float64
16  Pressure3pm          89780 non-null  float64
17  Cloud9am             61944 non-null  float64
18  Cloud3pm             59514 non-null  float64
19  Temp9am              98902 non-null  float64
20  Temp3pm              97612 non-null  float64
21  RainToday            98537 non-null  object
22  RainTomorrow         99516 non-null  int64
dtypes: float64(16), int64(1), object(6)
memory usage: 17.5+ MB
```

```
In [10]: df.drop(columns=['Evaporation', 'Sunshine', 'Cloud9am', 'Cloud3pm', 'RainTomorrow']
# Remove invalid rows.
df = df.dropna()
```

```
In [11]: df
```

Out[11]:

	row ID	Location	MinTemp	MaxTemp	Rainfall	WindGustDir	WindGustSpe
0	Row0	Albury	13.4	22.9	0.6	W	4
1	Row1	Albury	7.4	25.1	0.0	WNW	4
2	Row2	Albury	17.5	32.3	1.0	W	4
3	Row3	Albury	14.6	29.7	0.2	WNW	5
4	Row4	Albury	7.7	26.7	0.0	W	3
...	...	...	...	...	...	...	...
99511	Row101816	Uluru	8.0	20.7	0.0	ESE	4
99512	Row101817	Uluru	3.5	21.8	0.0	E	3
99513	Row101818	Uluru	2.8	23.4	0.0	E	3
99514	Row101819	Uluru	3.6	25.3	0.0	NNW	2
99515	Row101820	Uluru	5.4	26.9	0.0	N	3

79140 rows × 18 columns



In [12]:

```

# Sort for consistent grouping
samples_sorted = df.sort_values(['Location']).copy()

# Create a 10-day group id per Location
samples_sorted['day_index'] = samples_sorted.groupby('Location').cumcount()
samples_sorted['ten_day_group'] = samples_sorted['day_index'] // 20

# Group by Location and 10-day group and take the mean of numeric columns
ten_day_avg = samples_sorted.groupby(['Location', 'ten_day_group']).mean(numeric

# Optional: drop intermediate columns
ten_day_avg = ten_day_avg.drop(columns=['day_index', 'ten_day_group'], errors='i

# Preview
print(ten_day_avg.head())

```

	Location	MinTemp	MaxTemp	Rainfall	WindGustSpeed	WindSpeed9am	\
0	Adelaide	7.950	17.465	0.95	36.05	10.30	
1	Adelaide	9.185	19.105	1.50	36.85	11.30	
2	Adelaide	10.455	21.415	0.56	37.85	13.75	
3	Adelaide	11.070	20.670	2.98	34.25	10.70	
4	Adelaide	10.935	18.565	2.56	38.30	11.80	

	WindSpeed3pm	Humidity9am	Humidity3pm	Pressure9am	Pressure3pm	Temp9am	\
0	14.85	57.80	45.65	1027.635	1025.565	12.810	
1	16.65	63.40	50.15	1021.140	1019.075	14.045	
2	16.45	51.00	39.10	1022.005	1018.895	16.235	
3	14.75	66.15	48.95	1019.935	1017.735	15.540	
4	16.65	73.05	55.60	1019.805	1017.770	14.210	

	Temp3pm
0	16.350
1	17.705
2	20.335
3	19.835
4	17.730

In [13]: `ten_day_avg`

Out[13]:

	Location	MinTemp	MaxTemp	Rainfall	WindGustSpeed	WindSpeed9am	Wind!
0	Adelaide	7.950	17.465	0.95	36.05	10.30	
1	Adelaide	9.185	19.105	1.50	36.85	11.30	
2	Adelaide	10.455	21.415	0.56	37.85	13.75	
3	Adelaide	11.070	20.670	2.98	34.25	10.70	
4	Adelaide	10.935	18.565	2.56	38.30	11.80	
...	...	...	...	...	...	...	...
3972	Woomera	19.465	32.030	2.06	48.40	22.25	
3973	Woomera	18.540	31.580	2.03	57.20	24.50	
3974	Woomera	11.115	24.545	0.71	47.45	23.90	
3975	Woomera	14.650	29.455	0.13	47.40	22.35	
3976	Woomera	16.285	30.810	0.06	49.35	25.45	

3977 rows × 13 columns



In [14]: `ten_day_avg['Location'].unique().tolist()`

```
Out[14]: ['Adelaide',
          'Albury',
          'AliceSprings',
          'BadgerysCreek',
          'Ballarat',
          'Bendigo',
          'Brisbane',
          'Cairns',
          'Canberra',
          'Cobar',
          'CoffsHarbour',
          'Dartmoor',
          'Darwin',
          'GoldCoast',
          'Hobart',
          'Katherine',
          'Launceston',
          'Melbourne',
          'MelbourneAirport',
          'Mildura',
          'Moree',
          'MountGambier',
          'Nhil',
          'NorahHead',
          'NorfolkIsland',
          'Nuriootpa',
          'PearceRAAF',
          'Perth',
          'PerthAirport',
          'Portland',
          'Richmond',
          'Sale',
          'Sydney',
          'SydneyAirport',
          'Townsville',
          'Tuggeranong',
          'Uluru',
          'WaggaWagga',
          'Walpole',
          'Watsonia',
          'Williamtown',
          'Witchcliffe',
          'Wollongong',
          'Woomera']
```

```
In [15]: ten_day_avg.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3977 entries, 0 to 3976
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  ---
0   Location               3977 non-null   object
1   MinTemp                3977 non-null   float64
2   MaxTemp                3977 non-null   float64
3   Rainfall               3977 non-null   float64
4   WindGustSpeed          3977 non-null   float64
5   WindSpeed9am           3977 non-null   float64
6   WindSpeed3pm           3977 non-null   float64
7   Humidity9am            3977 non-null   float64
8   Humidity3pm            3977 non-null   float64
9   Pressure9am            3977 non-null   float64
10  Pressure3pm            3977 non-null   float64
11  Temp9am                3977 non-null   float64
12  Temp3pm                3977 non-null   float64
dtypes: float64(12), object(1)
memory usage: 404.0+ KB
```

```
In [16]: import pandas as pd

# Step 1: Count the number of time steps per Location
location_counts = ten_day_avg['Location'].value_counts()

# Step 2: Identify Locations with at least 141 time steps
valid_locations = location_counts[location_counts >= 64].index

# Step 3: Filter to keep only valid Locations
df_valid = ten_day_avg[ten_day_avg['Location'].isin(valid_locations)].copy()

# Step 4: Trim each Location to the first 141 rows
df_trimmed = (
    df_valid.groupby('Location', group_keys=False)
    .apply(lambda x: x.iloc[:64])
    .reset_index(drop=True)
)

# Optional: Confirm result
print(df_trimmed['Location'].value_counts())
print("Total locations kept:", df_trimmed['Location'].nunique())
```

Adelaide	64
Albury	64
Nuriootpa	64
PearceRAAF	64
Perth	64
PerthAirport	64
Portland	64
Richmond	64
Sale	64
Sydney	64
SydneyAirport	64
Townsville	64
Tuggeranong	64
WaggaWagga	64
Walpole	64
Watsonia	64
Williamtown	64
Witchcliffe	64
Wollongong	64
NorfolkIsland	64
NorahHead	64
MountGambier	64
Cobar	64
AliceSprings	64
BadgerysCreek	64
Ballarat	64
Bendigo	64
Brisbane	64
Cairns	64
Canberra	64
CoffsHarbour	64
Moree	64
Dartmoor	64
Darwin	64
GoldCoast	64
Hobart	64
Melbourne	64
MelbourneAirport	64
Mildura	64
Woomera	64

Name: Location, dtype: int64
Total locations kept: 40

```
In [17]: df_trimmed.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2560 entries, 0 to 2559
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Location               2560 non-null   object
1   MinTemp                2560 non-null   float64
2   MaxTemp                2560 non-null   float64
3   Rainfall               2560 non-null   float64
4   WindGustSpeed          2560 non-null   float64
5   WindSpeed9am           2560 non-null   float64
6   WindSpeed3pm           2560 non-null   float64
7   Humidity9am            2560 non-null   float64
8   Humidity3pm            2560 non-null   float64
9   Pressure9am            2560 non-null   float64
10  Pressure3pm            2560 non-null   float64
11  Temp9am                2560 non-null   float64
12  Temp3pm                2560 non-null   float64
dtypes: float64(12), object(1)
memory usage: 260.1+ KB
```

```
In [18]: # Filter to only observations from a list of locations.
#Locations = ['Brisbane', 'Melbourne', 'Sydney'] # your list of locations
#Locations = ['Brisbane'] # your list of locations
locations = df_trimmed['Location'].unique().tolist()
samples = df_trimmed[df_trimmed['Location'].isin(locations)]

# Remove invalid rows.
samples = samples.dropna()

# Show information about the data.
print(samples.info())
samples.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 2560 entries, 0 to 2559
Data columns (total 13 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Location               2560 non-null   object
1   MinTemp                2560 non-null   float64
2   MaxTemp                2560 non-null   float64
3   Rainfall               2560 non-null   float64
4   WindGustSpeed          2560 non-null   float64
5   WindSpeed9am           2560 non-null   float64
6   WindSpeed3pm           2560 non-null   float64
7   Humidity9am            2560 non-null   float64
8   Humidity3pm            2560 non-null   float64
9   Pressure9am            2560 non-null   float64
10  Pressure3pm            2560 non-null   float64
11  Temp9am                2560 non-null   float64
12  Temp3pm                2560 non-null   float64
dtypes: float64(12), object(1)
memory usage: 280.0+ KB
None
```

Out[18]:

	MinTemp	MaxTemp	Rainfall	WindGustSpeed	WindSpeed9am	WindSp
count	2560.000000	2560.000000	2560.000000	2560.000000	2560.000000	256
mean	12.607527	23.487586	2.432283	40.745137	15.147949	1
std	5.053111	5.383574	2.858874	5.898471	4.239553	
min	-0.675000	9.680000	0.000000	21.550000	3.900000	
25%	8.865000	19.503750	0.670000	36.950000	12.150000	1
50%	12.102500	23.237500	1.580000	40.700000	15.100000	1
75%	16.250000	27.442500	3.130000	44.550000	17.950000	2
max	25.640000	39.810000	37.440000	63.850000	28.250000	3



In [19]:

samples

Out[19]:

	Location	MinTemp	MaxTemp	Rainfall	WindGustSpeed	WindSpeed9am	WindSp
0	Adelaide	7.950	17.465	0.95	36.05	10.30	
1	Adelaide	9.185	19.105	1.50	36.85	11.30	
2	Adelaide	10.455	21.415	0.56	37.85	13.75	
3	Adelaide	11.070	20.670	2.98	34.25	10.70	
4	Adelaide	10.935	18.565	2.56	38.30	11.80	
...	...	...	...	...	...	...	
2555	Woomera	5.465	16.585	0.57	36.50	10.85	
2556	Woomera	9.100	20.750	0.18	43.65	18.80	
2557	Woomera	10.450	21.885	2.30	42.20	16.85	
2558	Woomera	11.860	24.495	0.01	36.25	16.20	
2559	Woomera	8.770	20.980	0.38	40.90	15.50	

2560 rows × 13 columns



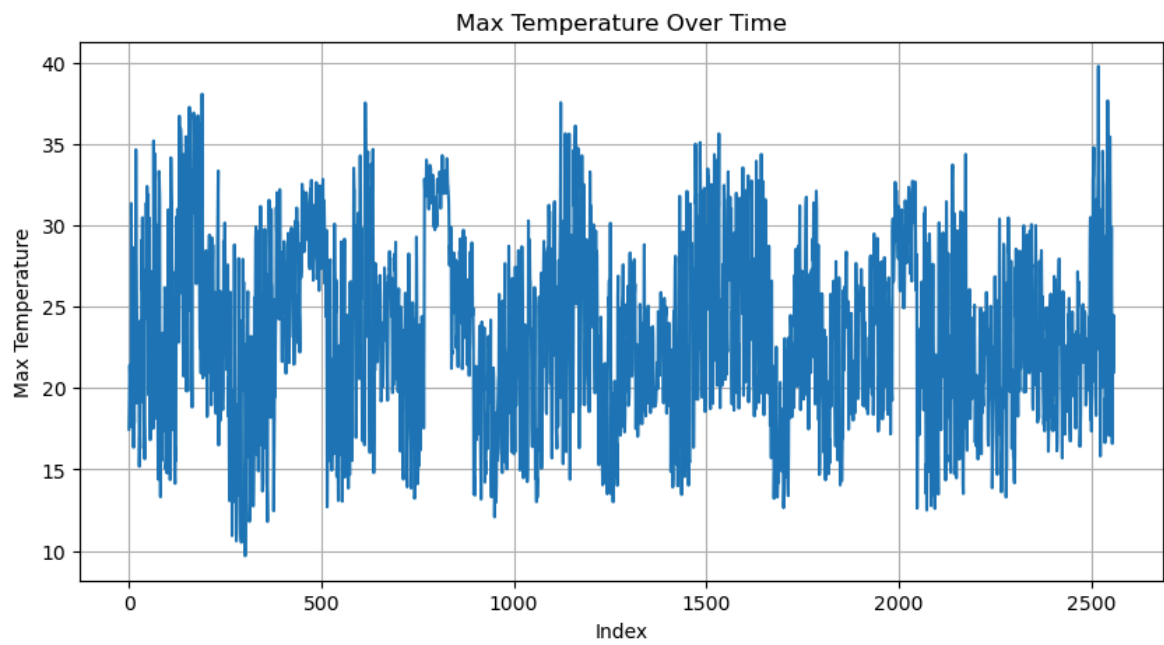
In [20]:

samples.to_csv('weather_data.csv', index=False)

In [21]:

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.plot(samples['MaxTemp'])
plt.title('Max Temperature Over Time')
plt.xlabel('Index')
plt.ylabel('Max Temperature')
plt.grid(True)
plt.show()
```



```
In [22]: print(samples['Location'].value_counts())
```

```

Adelaide          64
Albury            64
Nuriootpa         64
PearceRAAF        64
Perth             64
PerthAirport      64
Portland          64
Richmond          64
Sale              64
Sydney            64
SydneyAirport     64
Townsville        64
Tuggeranong       64
WaggaWagga        64
Walpole           64
Watsonia          64
Williamtown       64
Witchcliffe       64
Wollongong        64
NorfolkIsland     64
NorahHead         64
MountGambier      64
Cobar             64
AliceSprings      64
BadgerysCreek     64
Ballarat          64
Bendigo           64
Brisbane          64
Cairns            64
Canberra          64
CoffsHarbour      64
Moree             64
Dartmoor          64
Darwin            64
GoldCoast         64
Hobart            64
Melbourne         64
MelbourneAirport  64
Mildura           64
Woomera           64
Name: Location, dtype: int64

```

```
In [23]: samples.columns
```

```

Out[23]: Index(['Location', 'MinTemp', 'MaxTemp', 'Rainfall', 'WindGustSpeed',
               'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',
               'Pressure9am', 'Pressure3pm', 'Temp9am', 'Temp3pm'],
              dtype='object')

```

Univariate Forecasting

Next, we need to select the features that we are going to use as inputs for our weather forecast. Whether we select a single feature, or multiple features will determine whether the model is a univariate or multivariate LSTM.

Univariate LSTMs process a single time-dependent variable, such as daily stock prices or temperature readings, focusing on temporal patterns within that single feature to predict

future values of the same variable. In contrast, multivariate LSTMs handle multiple interrelated variables simultaneously, such as temperature, humidity, wind speed, and pressure for weather forecasting, allowing the model to capture not only temporal dependencies but also complex relationships between different features over time.

To begin with, we will start with a simple, univariate model. We will use the historical (`MaxTemp`) as the time-dependent variable, and predict the maximum temperature for the day (`MaxTemp`). Extract this feature into the corresponding X/y values as we have done in previous practicals.

Creating Sequences for Forecasting Multiple Data Points

We cannot simply feed the input data into the network in its original row-based form as we have done with other network architectures. Since we are predicting the "next" multiple time series value based on previous observations, we need to first transform the data into a "sequence", where each element in the sequence represents the attribute at that given point in time.

This is also known as sequence-to-sequence prediction. Sequence-to-sequence is a more challenging problem, as we have to forecast a greater number of data points with the same amount of input data, compared to predicting one value at a time, where you have another time step worth of features.

To do so, we are going to define a function named `create_sequences()`. This function should take our input data, the target values, and a sequence length as parameters. Our input is going to be of dimension $N \times T \times F$, and create an output of $N \times P$, where N represents the number of samples, T represents the number of time points to lookback in the past, F represents the number of features, and P represents the number of future time points (forecast horizon) that we are predicting.

Note that all sequences are padded to the same length, which ensures that the model can efficiently handle inputs of varying original lengths while maintaining consistent input dimensions.

```
In [24]: def create_sequences(sequence, lookback, forecast_horizon, target_col):
    T, num_features = sequence.shape
    X, y, lengths = [], [], []
    pad_vector = np.zeros((lookback, num_features))

    # Fixed-Length Lookback with pre-padding
    for t in range(1, T - forecast_horizon + 1):
        context = sequence[:t]
        if len(context) > lookback:
            context = context[-lookback:]

        padded_context = pad_vector.copy()
        padded_context[-len(context):] = context

        X.append(padded_context)
        y.append(sequence[t:t + forecast_horizon, target_col])
```

```
lengths.append(min(len(context), lookback))

return np.array(X), np.array(y), lengths
```

The `data_prep` processes each location separately, splitting the data into training and testing sets, where the last `test_steps` are reserved for testing. We will standardize the feature by removing the mean and scaling to unit variance with the scikit-learn `StandardScaler` class. Normalizing the data will improve the performance of our LSTM model. The sequences created using `create_sequences()` are collected across all locations and concatenated into arrays suitable for LSTM input. Storing the `train_sample` and `test_sample` will allow us to evaluate the model.

```
In [25]: import numpy as np
import torch
from sklearn.preprocessing import StandardScaler

def data_prep(df, feature_columns, lookback, test_steps, target_col):
    # Prepare to store all training data
    X_all, y_all = [], []
    location_ids = [] # To track which location each sample comes from
    test_data = [] # To store test data for each location
    train_data = []
    lengths_all = [] # To store the lengths of each sequence for packing

    # Process each Location's data
    for location_id, group in df.groupby('Location'):
        feature_values = group[feature_columns].values

        # If there are not enough samples for training, skip this location
        if len(feature_values) <= 63:
            continue

        # Separate the last 12 steps for testing
        # Scale the data
        scaler = StandardScaler()
        train_sample = scaler.fit_transform(feature_values[:-test_steps])
        test_sample = scaler.transform(feature_values[-test_steps:])

        # If you prefer without scaling
        # train_sample = feature_values[:-test_steps]
        # test_sample = feature_values[-test_steps:]

        train_data.append((location_id, train_sample))
        test_data.append((location_id, test_sample, scaler)) # Store test data

        # Prepare LSTM sequence data for training
        X_location, y_location, lengths = create_sequences(train_sample, lookback)

        # Append to the overall dataset
        X_all.append(X_location)
        y_all.append(y_location)
        lengths_all.append(lengths) # Store sequence lengths

        # Store Location ID for tracking
        location_ids.extend([location_id] * len(y_location))

    # Concatenate all locations' training data for model training
```

```

X_all = np.concatenate(X_all, axis=0)
y_all = np.concatenate(y_all, axis=0)
lengths_all = np.concatenate(lengths_all, axis=0) # Concatenate sequence lengths

# Reshape X to be [samples, time steps, features] as required by LSTM
X_all = X_all.reshape((X_all.shape[0], X_all.shape[1], X_all.shape[2]))

return torch.Tensor(X_all), torch.Tensor(y_all), torch.Tensor(lengths_all),

```

We can then use this function to create our sequences for the `MaxTemp` data, and split it into the training/testing set as we have done in previous practicals. For this demonstration, we will use 5 time points in the past (`lookback`) to predict 5 time points in the future (`test_steps`). The `target_col` refers to the index of the target attribute for prediction. As this is a univariate model, the value `0` will represent the input as well as target.

```

In [26]: lookback = 5
test_steps = 5
target_col = 0
X_5, y_5, lengths_5, train_d_5, test_d_5 = data_prep(samples, ['MaxTemp'], lookback, test_steps, target_col)

print("Shape of input data after sequence creation:", X_5.shape)
print("Shape of targets after sequence creation:", y_5.shape)

```

```

Shape of input data after sequence creation: torch.Size([2160, 5, 1])
Shape of targets after sequence creation: torch.Size([2160, 5])

```

Defining the LSTM network

Pytorch includes the implementation of the long short-term memory layer in the `LSTM` class in the `nn` module. We can utilize this class in a new `nn.Module` network. Unlike the `Linear` class where we only had to define an input and output size, the `LSTM` class requires more parameters to be defined. These include:

- `input_size` : The number of features that are input into this layer.
- `hidden_size` : The number of features in the hidden layer. This also corresponds to the output size of the LSTM layer.
- `num_layers` : The number of recurrent stacked recurrent layers.
- `batch_first` : This parameter allows the input to be provided in the form of $N \times T \times F$. If we did not set `batch_first`, we would have to reverse the input tensor order, and provide the data in the form of $T \times N \times F$.

After the LSTM layer, we need to map the intermediate features to our output variable, which is a sequence of numerical values. Note that since we are predicting numerical values rather than a probability, we do not use an activation function on the output of the linear/mapping layer.

```

In [27]: import torch.nn as nn
from torch.nn.utils.rnn import pack_padded_sequence, pad_packed_sequence

class MyLSTMNet(nn.Module):

```

```

def __init__(self, num_features, hidden_layer_size, num_layers, output_size,
             super().__init__())
    self.lstm = nn.LSTM(input_size=num_features,
                        hidden_size=hidden_layer_size,
                        num_layers=num_layers,
                        batch_first=True)
    self.dropout = nn.Dropout(dropout_prob)
    self.fc = nn.Linear(hidden_layer_size, output_size)

def forward(self, data, lengths):
    # Pack the input sequence - This ensures that the LSTM ignores padded values
    packed_data = pack_padded_sequence(data, lengths.cpu(), batch_first=True)

    # Run through LSTM
    packed_output, (hn, cn) = self.lstm(packed_data)

    # Use the last layer's hidden state
    last_hidden = hn[-1]

    # Apply dropout and final linear layer
    out = self.dropout(last_hidden)
    out = self.fc(out)
    return out

```

Let us now initialize the necessary parameters for our LSTM model.

```

In [28]: num_features = X_5.shape[2]
         hidden_layer_size = 10
         output_size = test_steps
         num_layers = 2
         dropout_prob = 0.2
         model_lstm_5 = MyLSTMNet(num_features, hidden_layer_size, num_layers, output_size)

```

```

In [29]: print(model_lstm_5)

```

```

MyLSTMNet(
  (lstm): LSTM(1, 10, num_layers=2, batch_first=True)
  (dropout): Dropout(p=0.2, inplace=False)
  (fc): Linear(in_features=10, out_features=5, bias=True)
)

```

Defining the Training Process

As we are working with time series data, and predicting a numeric value rather than a class, we cannot use the previous `train_predict_model` function directly. Instead, we have to make a number of changes:

- Instead of computing accuracy, we will compute loss.
- Instead of using binary cross entropy as a loss function, we will use mean squared error. BCE loss only works on binary labels.
- Instead of displaying the loss after every epoch, we will only display it after every 100 epochs. Because of our selected dataset, the model will require significantly more epochs to train compared to the earlier neural networks we have worked with.
- Due to having a smaller number of samples, we will use a lower batch size to reduce the range of data that will affect the weights in one "pass". There is no "perfect"

batch size, the choice will depend both on your network design and dataset.

- We will return model that has lowest validation as the best model for evaluation/future usage.

Side note: Loss functions

Loss functions serve as quantitative measures of model performance in machine learning by calculating the disparity between predicted and actual values, essentially guiding optimization algorithms toward better parameter configurations.

```
In [30]: import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
import matplotlib.pyplot as plt
from torch.utils.data import DataLoader, TensorDataset, random_split

def train_predict_model(model, n_epochs, lr, X_all, y_all, lengths, validation_s
    batch_size = 32

    # Split data into train and validation sets
    dataset = TensorDataset(X_all, y_all, lengths)
    val_size = int(len(dataset) * validation_split)
    train_size = len(dataset) - val_size
    train_set, val_set = random_split(dataset, [train_size, val_size])

    train_loader = DataLoader(train_set, batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(val_set, batch_size=batch_size, shuffle=False)

    loss_fn = nn.MSELoss()
    optimizer = optim.Adam(model.parameters(), lr=lr)

    print(f"The model has {sum(p.numel() for p in model.parameters() if p.requires_grad_)} parameters")

    train_loss_history = []
    val_loss_history = []

    best_val_loss = float('inf')
    best_model_state = None

    for epoch in range(n_epochs):
        model.train()
        for X_batch, y_batch, lengths_batch in train_loader:
            y_pred = model(X_batch, lengths_batch)
            loss = loss_fn(y_pred, y_batch)
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        # Validation check every 100 epochs
        if epoch % 100 == 0:
            model.eval()
            with torch.no_grad():
                train_preds = model(X_all[train_set.indices], lengths[train_set.indices])
                train_loss = loss_fn(train_preds, y_all[train_set.indices]).item()
```

```

val_preds = model(X_all[val_set.indices], lengths[val_set.indices])
val_loss = loss_fn(val_preds, y_all[val_set.indices]).item()

print(f"Epoch {epoch+1}: train loss {train_loss:.4f}, val loss {

train_loss_history.append(train_loss)
val_loss_history.append(val_loss)

# Save best model
if val_loss < best_val_loss:
    best_val_loss = val_loss
    best_model_state = model.state_dict()

# Restore best model state
if best_model_state is not None:
    model.load_state_dict(best_model_state)

return train_loss_history, val_loss_history, model

```

Training the Univariate LSTM Model

We can now finally train our model. As mentioned above, we will need a significantly larger number of epochs for training this model compared to the others we have trained in this unit. Let's start with 500 epochs.

```

In [31]: n_epochs = 501
lr = 0.001
train_loss_history_5, val_loss_history_5, model_lstm_5 = train_predict_model(model, lr, vali

```

The model has 1,455 trainable parameters
Epoch 1: train loss 1.0146, val loss 0.9823
Epoch 101: train loss 0.8467, val loss 0.8303
Epoch 201: train loss 0.8100, val loss 0.8194
Epoch 301: train loss 0.7859, val loss 0.8140
Epoch 401: train loss 0.7643, val loss 0.8150
Epoch 501: train loss 0.7434, val loss 0.8189

As we did with the deep neural network, let's examine the performance across the training process by plotting epochs against loss.

```

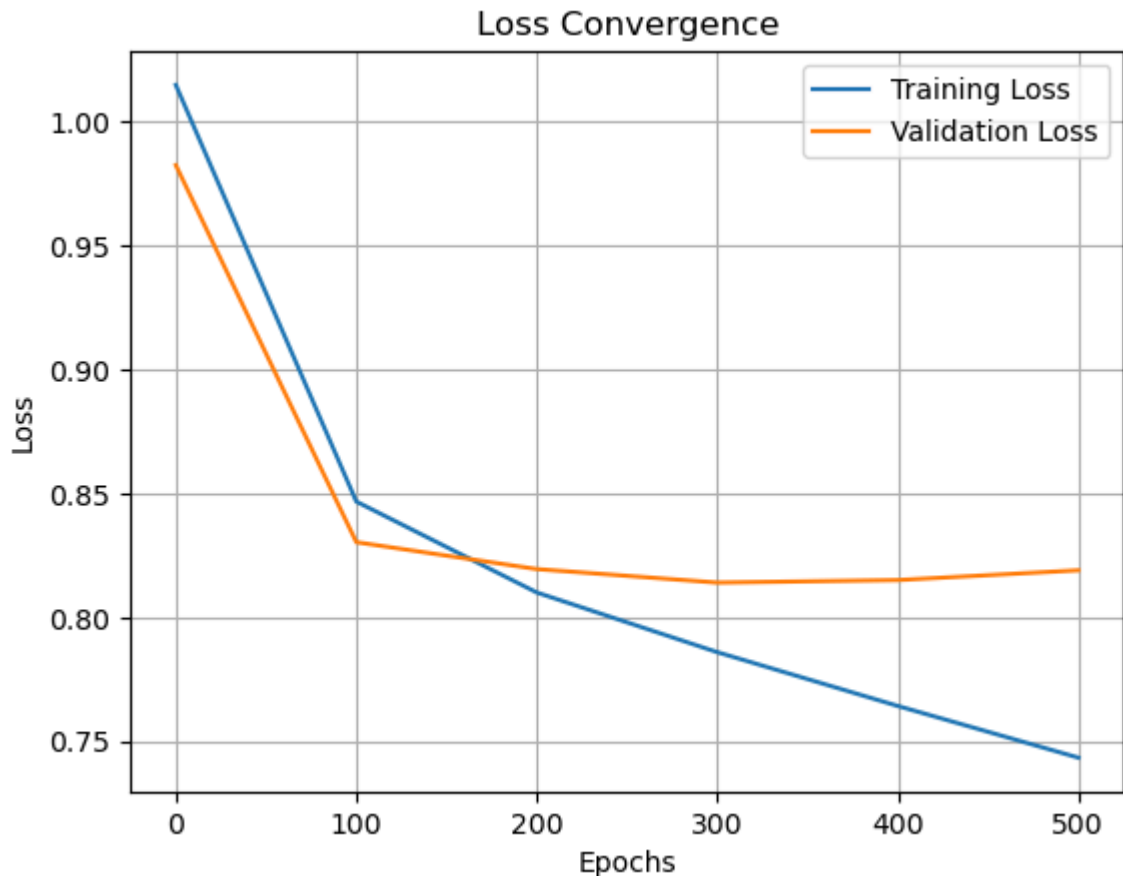
In [32]: def vis_train_loss(train_loss_history, val_loss_history):
    epochs = range(0, n_epochs, 100)
    plt.plot(epochs, train_loss_history, label='Training Loss')
    plt.plot(epochs, val_loss_history, label='Validation Loss') # <-- add this
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.title('Loss Convergence')
    plt.legend()
    plt.grid()
    plt.show()

```

```

In [33]: vis_train_loss(train_loss_history_5, val_loss_history_5)

```



We can observe that our training loss continued to decrease, but our testing loss started to increase above approximately 300. This would be another good place to utilize early stopping, but that is outside the scope of this practical document.

Computing the RMSE

To compute the root mean square error of the model's predictions, we can simply use `root_mean_squared_error` from scikit-learn.

We can compute the R^2 score in a similar manner.

Let's also plot the predictions that the LSTM model has made and compare them to the labelled data.

```
In [34]: from sklearn.metrics import root_mean_squared_error
from sklearn.metrics import r2_score
def pred_eval(model, X, y, lengths, train_d, test_d, lookback, target_col):
    model.eval()
    with torch.no_grad():
        train_preds = model(X, lengths)
        print("Training RMSE:", root_mean_squared_error(y.flatten().tolist(), train_preds.flatten().tolist()))
        print("Training R2:", r2_score(y.flatten().tolist(), train_preds.flatten().tolist()))
        X_test = []
        y_test = []
        lengths_test = []
        for count, (location_id, test_values, scaler) in enumerate(test_d):
            train_values = train_d[count][1]
            X_test.append(train_values[-lookback:])
```

```

y_test.append(test_values[:,target_col])
# Append the actual lengths (just like the training phase)
lengths_test.append(len(train_values[-lookback:]))
X_test = torch.Tensor(X_test)
y_test = torch.Tensor(y_test)
lengths_test = torch.Tensor(lengths_test).long()
test_preds = model(X_test,lengths_test)
#print(y_test.flatten().tolist())
#print(test_preds.flatten().tolist())
print("Test RMSE:",root_mean_squared_error(y_test.flatten().tolist(),tes
print("Test R2:", r2_score(y_test.flatten().tolist(),test_preds.flatten(
plt.figure(figsize=(10, 6))
plt.plot(y_test.flatten().tolist(), label="Expected Value")
plt.plot(test_preds.flatten().tolist(), label="Predicted Value")
plt.grid()
plt.legend(fontsize=10)
plt.tight_layout()
plt.show()

```

```

In [35]: lookback = 5
target_col = 0
pred_eval(model_lstm_5, X_5, y_5, lengths_5, train_d_5, test_d_5, lookback, targ

```

Training RMSE: 0.8709347585148683

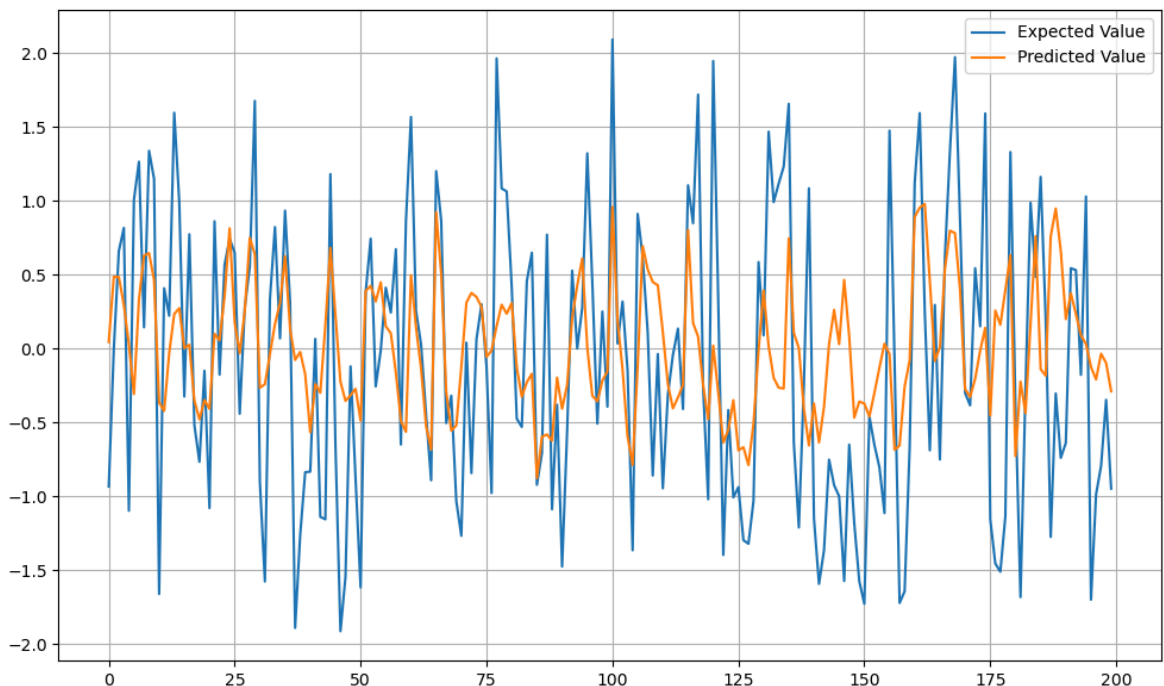
Training R2: 0.24242377731897058

Test RMSE: 0.8423998305810593

Test R2: 0.2319795473641233

C:\Users\balasubt\AppData\Local\Temp\ipykernel_47360\3342588267.py:18: UserWarning: Creating a tensor from a list of numpy.ndarrays is extremely slow. Please consider converting the list to a single numpy.ndarray with numpy.array() before converting to a tensor. (Triggered internally at C:\actions-runner_work\pytorch\pytorch\pytorch\torch\csrc\utils\tensor_new.cpp:257.)

```
X_test = torch.Tensor(X_test)
```



While we can see the general trend is being followed, it appears that our model is unable to predict many of the more extreme values.

From this plot, we can come to a conclusion: the model is incapable of accurately predicting the extreme values, but on average predicts in the correct ballpark. This may be due to a number of reasons: the model architecture, the data, or the fact that we are only making a prediction based on a single variable: the historical `MaxTemp`.

Multivariate LSTM model

Instead of only relying on the `MaxTemp`, let's utilize a larger number of the input features that we have available. Repeat the process from before by selecting the features, creating the sequence, and training the model. We do not need to change the architecture of the LSTM network, as we can simply provide a different number of input features to the constructor.

This gives us `12` features per time step. Now we can train the new model. Let us use the same `lookback` and `test_steps` values.

Note that the `target_col` is set to `1` because our target attribute, `MaxTemp`, is located at index `1` in the list of `12` features included in the model.

```
In [36]: lookback = 5
test_steps = 5
target_col = 1
X_5_mv, y_5_mv, lenghts_5_mv, train_d_5_mv, test_d_5_mv = data_prep(samples, ['Mi', 'Ra', 'Wi', 'Hu', 'Pr', 'Te'], lookback, test_steps)

print("Shape of input data after sequence creation:", X_5_mv.shape)
print("Shape of targets after sequence creation:", y_5_mv.shape)

num_features = X_5_mv.shape[2]
hidden_layer_size = 10
output_size = test_steps
n_epochs = 501
lr = 0.001
num_layers = 2
dropout_prob = 0.2

model_lstm_5_mv = MyLSTMNet(num_features, hidden_layer_size, num_layers, output_size)
train_loss_history_5_mv, val_loss_history_5_mv, model_lstm_5_mv = train_predict_model(model_lstm_5_mv, X_5_mv, y_5_mv, lenghts_5_mv, train_d_5_mv, test_d_5_mv)

vis_train_loss(train_loss_history_5_mv, val_loss_history_5_mv)
pred_eval(model_lstm_5_mv, X_5_mv, y_5_mv, lenghts_5_mv, train_d_5_mv, test_d_5_mv)
```

Shape of input data after sequence creation: torch.Size([2160, 5, 12])

Shape of targets after sequence creation: torch.Size([2160, 5])

The model has 1,895 trainable parameters

Epoch 1: train loss 0.9964, val loss 0.9915

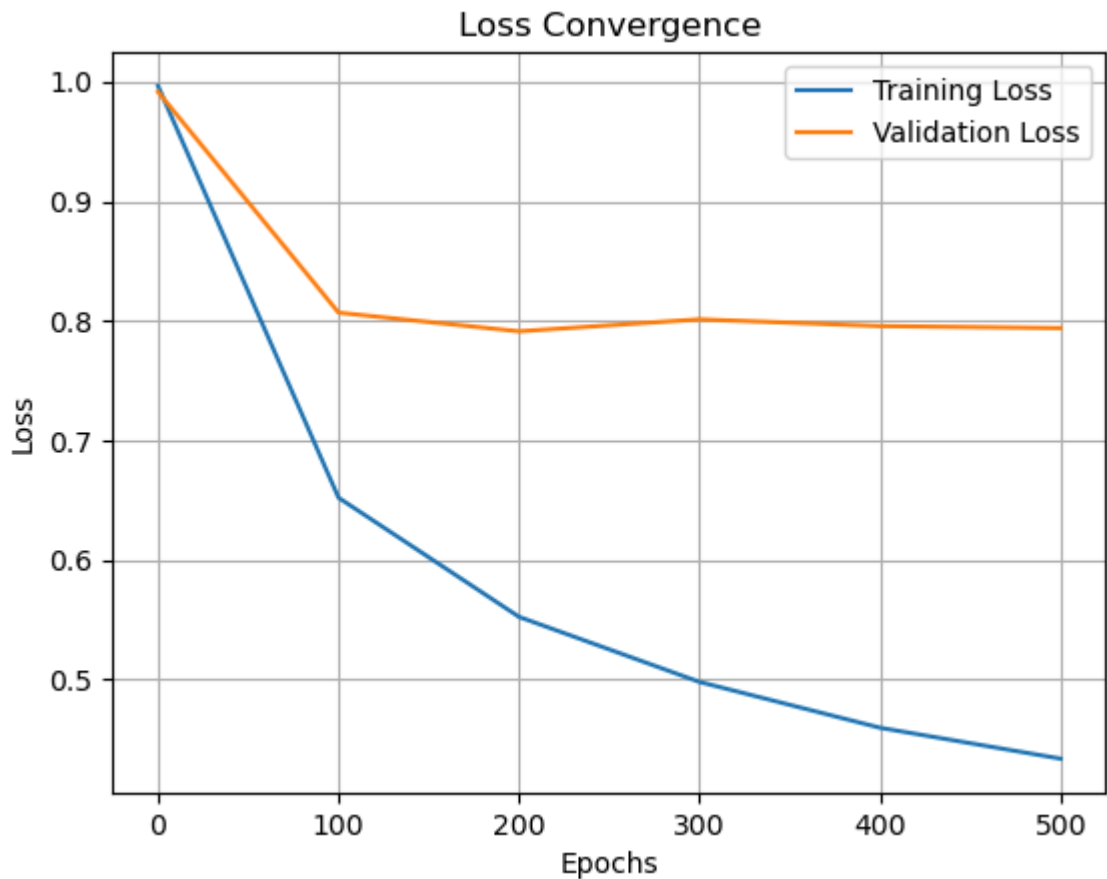
Epoch 101: train loss 0.6521, val loss 0.8068

Epoch 201: train loss 0.5525, val loss 0.7913

Epoch 301: train loss 0.4981, val loss 0.8013

Epoch 401: train loss 0.4598, val loss 0.7957

Epoch 501: train loss 0.4339, val loss 0.7940

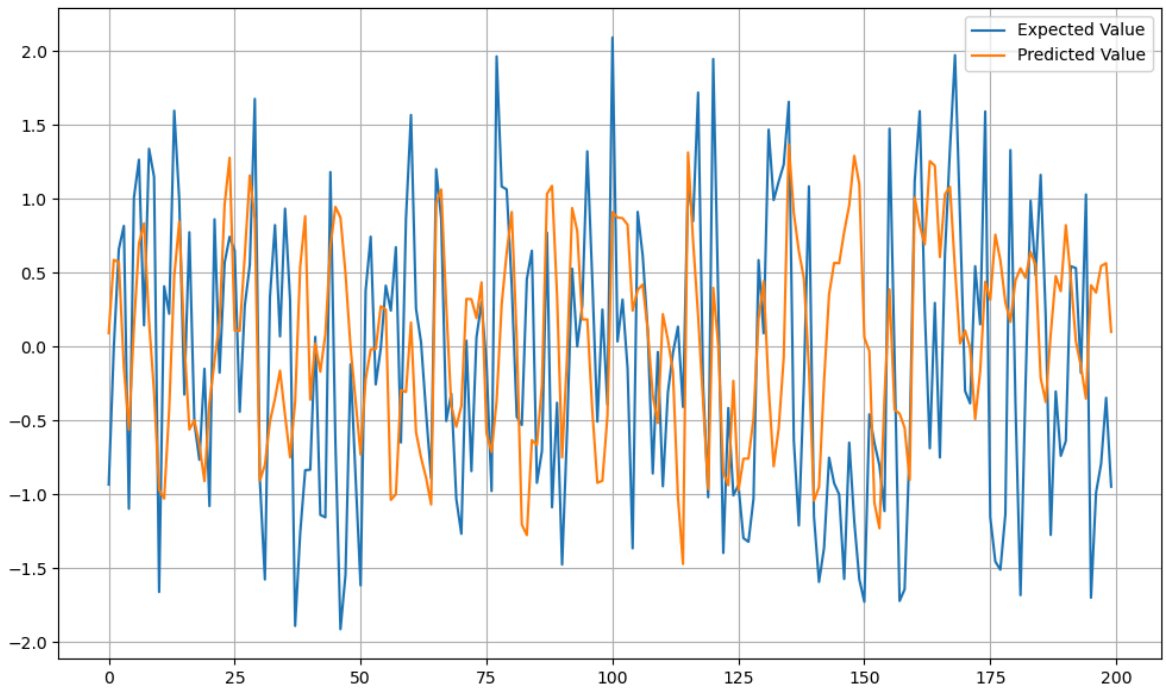


Training RMSE: 0.7112989879525925

Training R2: 0.4946881647185638

Test RMSE: 1.0400154065567497

Test R2: -0.1706195955480776



We can observe that the loss decreases significantly faster with the multivariate model, with the values for the loss itself are significantly lower. This would suggest that the multivariate model is both making better predictions and retaining information from the training set.

With the multivariate model, we have reached a loss of `~0.43` for training and `~0.79` for validation, which means the model is fitting well to the training data and has overfit.

Now, let us increase the lookback to `10` and see how the model performance changes.

```
In [37]: lookback = 10
test_steps = 5
target_col = 1
X_10_mv, y_10_mv, lengths_10_mv, train_d_10_mv, test_d_10_mv = data_prep(samples

lookback

print("Shape of input data after sequence creation:", X_10_mv.shape)
print("Shape of targets after sequence creation:", y_10_mv.shape)

num_features = X_10_mv.shape[2]
hidden_layer_size = 10
output_size = test_steps
n_epochs = 500
lr = 0.001
num_layers = 2
dropout_prob = 0.2
model_lstm_10_mv = MyLSTMNet(num_features, hidden_layer_size, num_layers, output

train_loss_history_10_mv, val_loss_history_10_mv, model_lstm_10_mv = train_predic
```

```
vis_train_loss(train_loss_history_10_mv, val_loss_history_10_mv)
```

```
pred_eval(model_lstm_10_mv, X_10_mv, y_10_mv, lengths_10_mv, train_d_10_mv, test
```

Shape of input data after sequence creation: torch.Size([2160, 10, 12])

Shape of targets after sequence creation: torch.Size([2160, 5])

The model has 1,895 trainable parameters

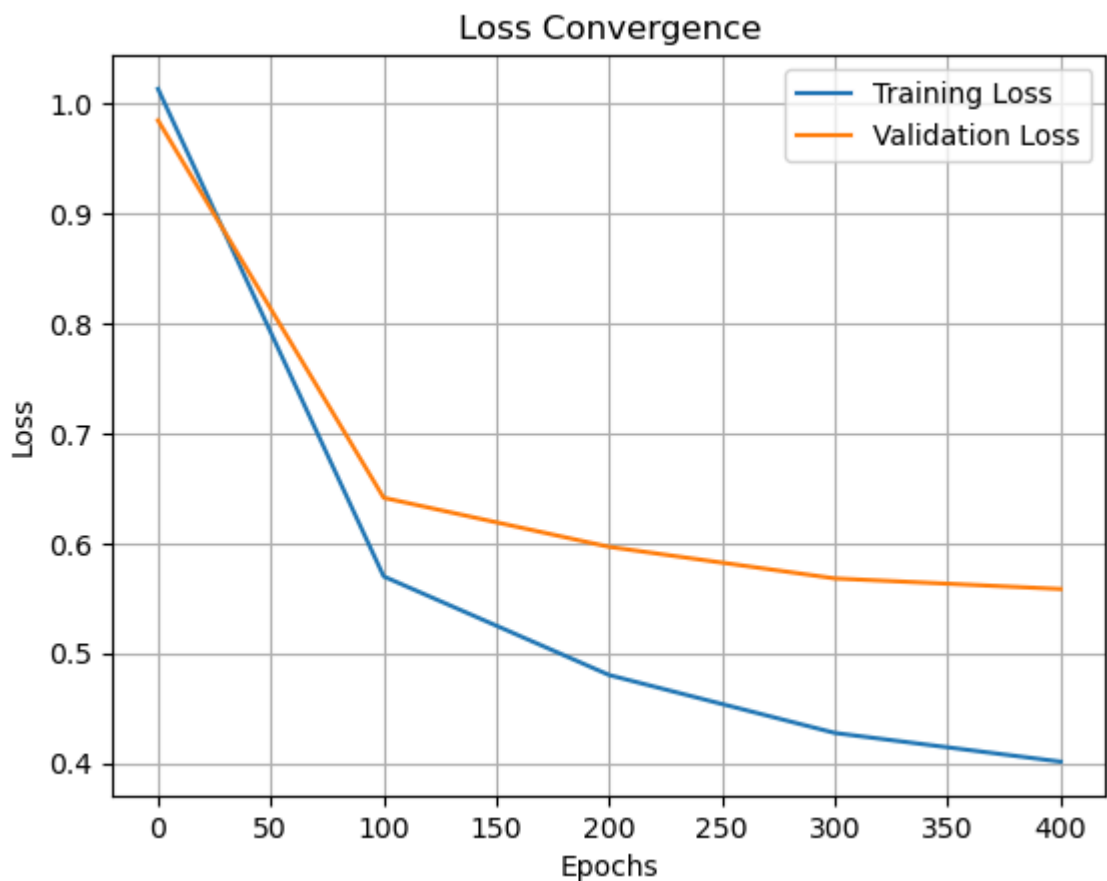
Epoch 1: train loss 1.0135, val loss 0.9849

Epoch 101: train loss 0.5699, val loss 0.6414

Epoch 201: train loss 0.4801, val loss 0.5967

Epoch 301: train loss 0.4274, val loss 0.5680

Epoch 401: train loss 0.4013, val loss 0.5584

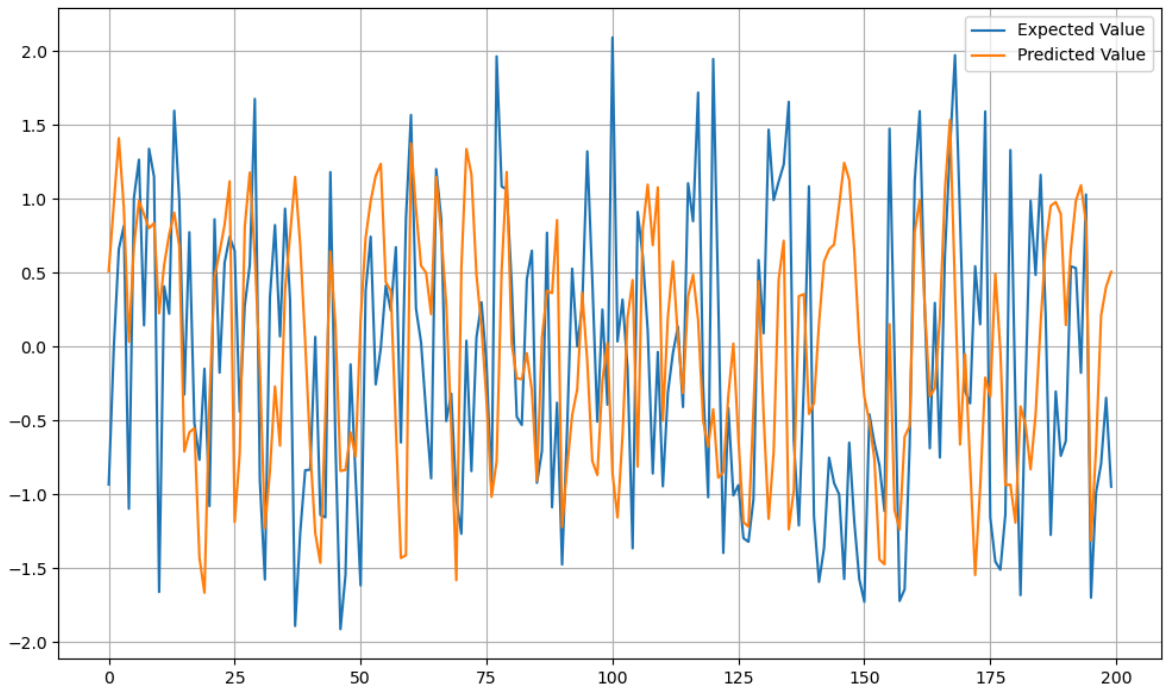


Training RMSE: 0.6414204968166047

Training R2: 0.5890956334201296

Test RMSE: 1.1004811789559064

Test R2: -0.31069451722422414



So far, we have explored how to build univariate and multivariate models with a fixed lookback. Now, let us see how to adapt the LSTM model to support a variable lookback.

To achieve that we need to make changes to the `create_sequences` function. For simplicity, let us consider a lookback value of `0` as varying lookback. If lookback is set to `0`, the function uses the full available history up to each time step, resulting in variable-length inputs, which are padded automatically using PyTorch's `pad_sequence` for batch processing. The function returns the padded input sequences, corresponding targets, and a list of sequence lengths for use with LSTM models.

```
In [38]: import numpy as np
import torch
import torch.nn.functional as F
from torch.nn.utils.rnn import pad_sequence

def create_sequences(sequence, lookback, forecast_horizon, target_col, pad_value):
    T, num_features = sequence.shape
    X, y, lengths = [], [], []

    if lookback > 0:
        # Fixed-length lookback with pre-padding
        pad_vector = np.zeros((lookback, num_features))

        for t in range(1, T - forecast_horizon + 1):
            context = sequence[:t]
            if len(context) > lookback:
                context = context[-lookback:]

            padded_context = pad_vector.copy()
            padded_context[-len(context):] = context

            X.append(padded_context)
            y.append(sequence[t:t + forecast_horizon, target_col])
            lengths.append(min(len(context), lookback))

    return np.array(X), np.array(y), lengths
```

```

else:
    # Full history with left pre-padding (variable-length inputs)
    for t in range(1, T - forecast_horizon + 1):
        context = torch.tensor(sequence[:t], dtype=torch.float32)
        lengths.append(t)

        X.append(context) # No manual padding
        y.append(torch.tensor(sequence[t:t + forecast_horizon, target_col],

X_padded = pad_sequence(X, batch_first=True, padding_value=pad_value)
y_tensor = torch.stack(y)

return X_padded.numpy(), y_tensor.numpy(), lengths

```

```

In [39]: lookback = 0 # Means no restriction of lookback
target_col = 1
X_f, y_f, lengths_f, train_d_f, test_d_f = data_prep(samples, ['MinTemp', 'MaxTemp',
    'WindSpeed9am', 'WindSpeed3pm', 'Humidity9am', 'Humidity3pm',
    'Pressure9am', 'Pressure3pm', 'Temp9am', 'Temp3pm'], lookback, test_steps,

print("Shape of input data after sequence creation:", X_f.shape)
print("Shape of targets after sequence creation:", y_f.shape)

num_features = X_f.shape[2]
hidden_layer_size = 10
output_size = test_steps
n_epochs = 500
lr = 0.001
num_layers = 2
dropout_prob = 0.2
model_lstm_f = MyLSTMNet(num_features, hidden_layer_size, num_layers, output_size

train_loss_history_f, val_loss_history_f, model_lstm_f = train_predict_model(model
    lr, X
    valid

vis_train_loss(train_loss_history_f, val_loss_history_f)

pred_eval(model_lstm_f, X_f, y_f, lengths_f, train_d_f, test_d_f, lookback, targ

```

Shape of input data after sequence creation: torch.Size([2160, 54, 12])

Shape of targets after sequence creation: torch.Size([2160, 5])

The model has 1,895 trainable parameters

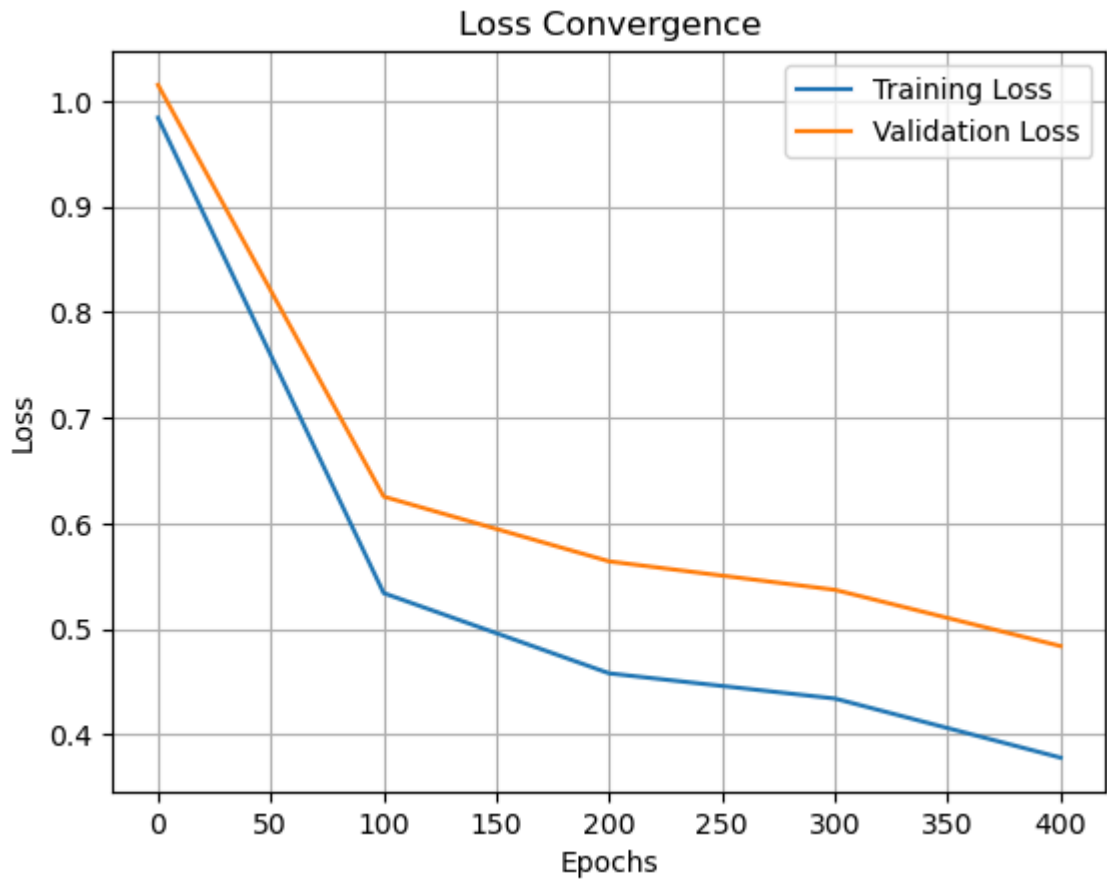
Epoch 1: train loss 0.9840, val loss 1.0151

Epoch 101: train loss 0.5340, val loss 0.6254

Epoch 201: train loss 0.4580, val loss 0.5640

Epoch 301: train loss 0.4341, val loss 0.5371

Epoch 401: train loss 0.3782, val loss 0.4838

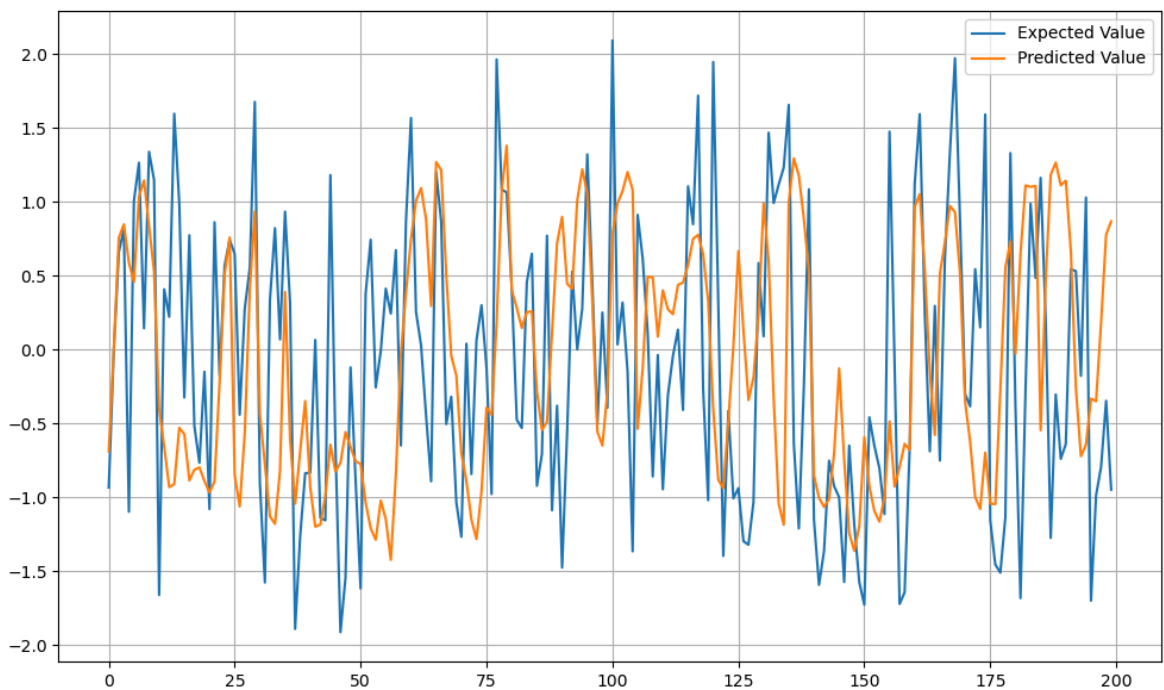


Training RMSE: 0.6073844507202192

Training R2: 0.6315467191801275

Test RMSE: 1.0568317448394302

Test R2: -0.2087818877948151



While some of the more extreme values are still mispredicted, we can see that the predicted temperatures are much closer to the actual values on unseen data. This shows that for this particular dataset, using multiple variables can provide a significant performance improvement.

One can also notice that the R-squared value is negative for the test dataset. This is quite common when a small number of test samples are considered, as in our case. Therefore, it is advisable not to rely heavily on the R-squared value and to focus more on RMSE or other similar metrics.

Other aspects that could potentially further improve performance, but are not included in this practical include:

- Using the categorical fields such as wind speed, by e.g. one-hot encoding them.
- Tuning the number of time points/sequence length used to make a prediction (we set it to 5 and 10).
- Tuning the batch size hyperparameter.
- Tuning the number of layers in the LSTM model.
- Tuning the size of the hidden layer in the LSTM model.
- Using additional hidden layers following the LSTM layer and creating a deeper network.

End notes

In this practical, we learned how to build deep neural networks using the `Sequential` class in Pytorch, and compared the performance with a single hidden layer neural network. We found that on the Veteran dataset, using a deep network marginally improved the performance.

We also introduced recurrent neural networks by implementing a network that utilizes long short-term memory cells, or LSTMs. We implemented both univariate (single time dependent variable) and multivariate (multiple time dependent variables) models, and found that for the dataset we used and the weather forecasting application, multivariate was a significant improvement.

Finally, we introduced time series forecasting with both fixed and variable lookback windows.